SAGAR

Government Subsidy & Grant Administration Platform

PROJECT DOCUMENTATION

Final Project Report | All Four Milestones

Team: Mahesh • Kasturi • Akshaya

1. Project Overview

SAGAR is designed to manage a subsidy application from beneficiary registration through scheme selection, eligibility assessment, verification, approval, disbursement, compliance monitoring and reporting. It brings these activities into a connected digital workflow so applicants and authorized officers can work with a shared record rather than disconnected steps.

1.1 Problem statement: a beneficiary story

Consider Savitri, a fictional woman entrepreneur in a rural community. She learns about a women's entrepreneurship scheme that may help her establish a small business. She must determine whether her age and income fit the scheme, identify required documents, submit an application and wait for a decision. If a document is missing or an officer requests clarification, she may struggle to understand what needs to happen next. Without a clear status trail, it is difficult to know whether the application is waiting for verification, district review or finance approval.

This is an illustrative scenario, not a claim about a specific real person. It represents the process challenges the project aims to address: unclear eligibility rules, document-related delays, fragmented verification, limited application visibility and weak end-to-end traceability.

1.2 Objectives

- Centralize beneficiary profiles, scheme definitions, applications and supporting evidence.
- Provide scheme-aware eligibility checks and a transparent 100-point score where configured.
- Route cases through field verification, district review, finance approval and disbursement roles.
- Track staged releases, compliance evidence, reminders, audit history and reporting information.
- Use role-based access so users see actions relevant to their responsibilities.

1.3 Scope

The project is implemented as a Java/Spring web application with a persistent data layer. External production integrations, deployment hardening and live treasury connectivity are environment-dependent until separately configured and validated.

2. Team Members and Contributions

The team worked across the beneficiary experience, scheme/eligibility logic and workflow implementation. The contribution descriptions below are a proposed presentation-ready summary; confirm them against the team's actual task allocation before formal submission.

Team member	Contribution area	Summary
Mahesh	Frontend and user experience	Beneficiary/officer screens, form layout, dashboards, user-facing status and application journey.

Kasturi	Scheme rules and eligibility	Scheme criteria, required evidence definitions, scoring requirements and rule-focused validation.
Akshaya	Beneficiary and workflow	Beneficiary profile/application journey, verification-stage coordination, resubmission flow and workflow integration.

2.1 Team integration

The interface collects data and presents status; backend services validate profile and scheme rules; workflow services manage authorized transitions; persistence stores application and evidence records; and dashboards summarize activity.

3. Technology Stack

	Technology	Purpose
Runtime	Java 21	Application implementation and runtime baseline.
Backend	Spring Boot	Configuration, dependency injection and web services.
Build	Maven / Maven Wrapper	Repeatable builds and dependency resolution.
Persistence	Spring Data JPA / Hibernate	Entity mapping and database operations.
Database	MySQL	Persistent storage for workflow data.
Security	Spring Security / BCrypt (as configured)	Authentication and role-aware access control.
Presentation	Server-rendered templates / web controllers	Forms, dashboards and role-specific views.
API	REST endpoints where implemented	Programmatic access to scheme, officer and application functions.

4. Environment Setup

4.1 Prerequisites

- JDK 21 or the version specified by project configuration.
- MySQL Server and a database/schema for the application.
- Maven or the project Maven Wrapper.
- IDE with Java support and a modern web browser.

4.2 Setup procedure

1. Extract the project ZIP and open the project root in an IDE.
2. Inspect the README, Maven configuration and application configuration files.
3. Create the MySQL database and a dedicated database user with minimum required permissions.
4. Configure datasource URL, username, password and active profile values; do not commit production secrets.
5. Run the project tests and build using the Maven Wrapper where available.
6. Start the Spring Boot application using the documented run configuration or main application class.
7. Open the configured local URL and verify login, scheme listing, beneficiary profile and application workflow.

4.3 Configuration and safety

Exact database names, ports, credentials and deployment URLs must come from the local environment rather than being assumed. Keep secrets outside source control, use separate development/test/production settings, and verify schema initialization before using existing data.

5. System Architecture

The platform follows a layered web-application structure. The presentation layer collects input and renders status; controllers accept requests; service classes apply eligibility and workflow rules; repositories persist records; and MySQL provides durable storage.

- Presentation: beneficiary pages and role-specific officer/admin views.
- Controllers: request handling, form submission, eligibility checks and workflow actions.
- Services: scheme validation, scoring, verification routing, sanction/disbursement and compliance operations.
- Persistence: repositories and entities for users, beneficiaries, schemes, applications, documents and audit records.
- Infrastructure: Java 21, Spring Boot, Spring Security, JPA/Hibernate, MySQL and Maven.

5.1 Cross-cutting controls

Authentication, authorization, input validation, audit logging and exception handling apply across the workflow. Role and jurisdiction checks should be enforced server-side, not only by hiding interface elements. Audit records should explain who performed a state-changing action, when it happened and what changed.

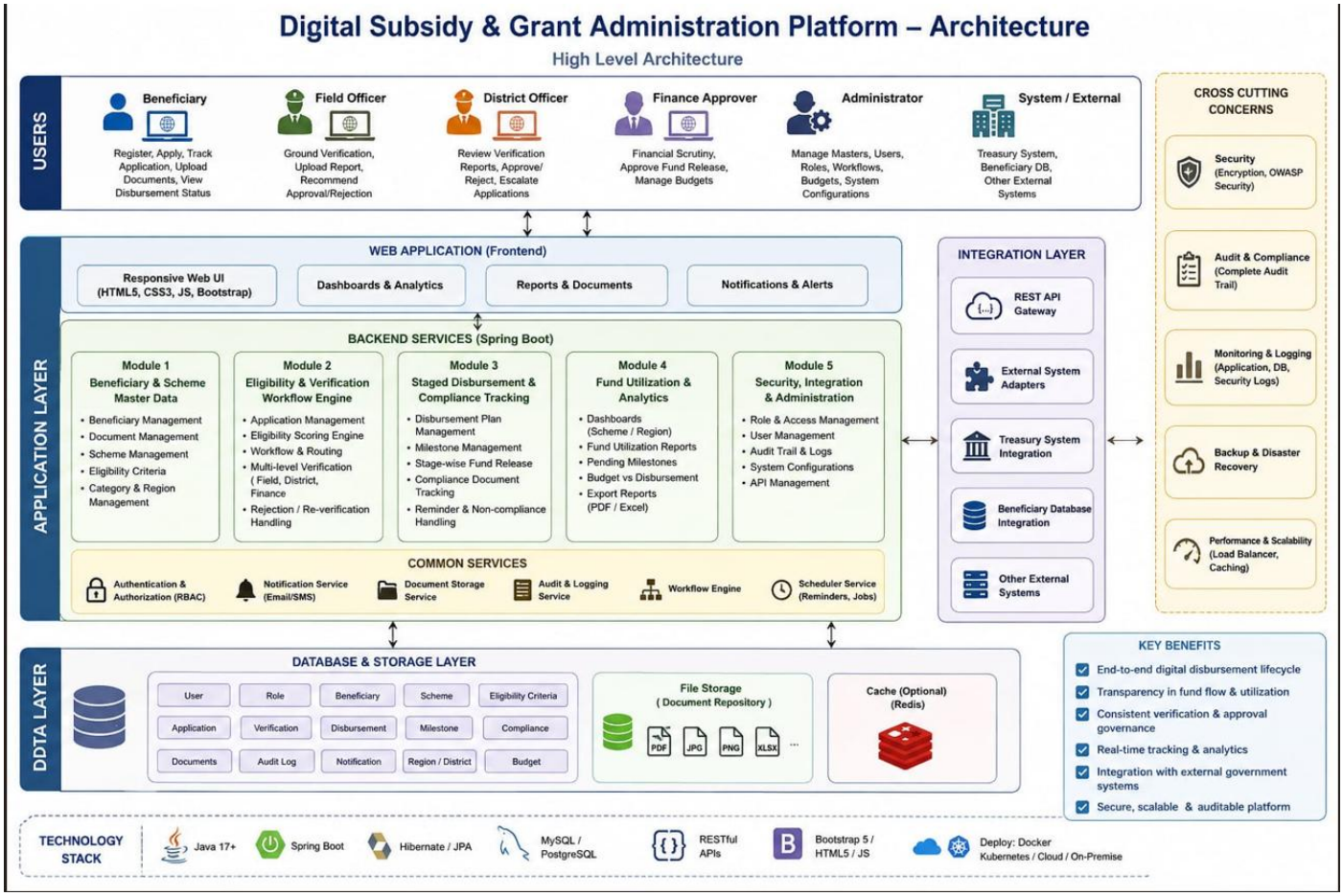


Figure 2. Conceptual architecture of the Java/Spring application.

6. Beneficiary and Scheme Data Models

6.1 Beneficiary and user data

The beneficiary profile stores identity and attributes required for eligibility checks and applicant contact. Depending on the configured schema, these may include government ID, contact information, age/date of birth, income, occupation, gender, disability status and residence. Authentication data belongs to the user/security model; sensitive values should be protected and shown only to authorized roles.

6.2 Scheme master data

A scheme record represents eligibility gates, age and income limits, scheme-specific attributes, required/optional documents and any regional budget information supported by the implementation.

Scheme	Eligibility dimensions	Example evidence
Education Subsidy	Age, income, student status	Student ID / education evidence
Farmer Support Grant	Age, income, farmer status	Land record / KCC or farming evidence
Housing Assistance	Age, income, housing conditions	Property / housing evidence
Women Entrepreneurship	Age, income, female applicant, business purpose	Business plan and identity evidence
Senior Citizen Assistance	Age and income	Age and income evidence
Divyang Disability Support	Age, income, disability status	UDID / disability evidence
Rural Skill Development	Age, income, rural residence	Residence / training evidence

6.3 Application and document records

An application links a beneficiary to a scheme and stores submitted details, eligibility score/status, current workflow state, officer remarks, sanction information and resubmission metadata. Document records connect evidence files to the relevant application and should capture document type, upload status and verification outcome.

7. Eligibility Scoring Logic

The project describes a 100-point scoring model made up of profile attributes and document evidence. The score supports a structured assessment; it does not replace mandatory scheme gates or an authorized officer's required review.

Component	Maximum points	Purpose
Beneficiary profile	50	Configured profile criteria such as age, income, government ID, occupation, gender, disability and residence.
Mandatory documents	40	Completion/validation of required scheme evidence.
Optional documents	10	Additional supporting evidence where configured.
Total	100	Common score format for applicant feedback and review.

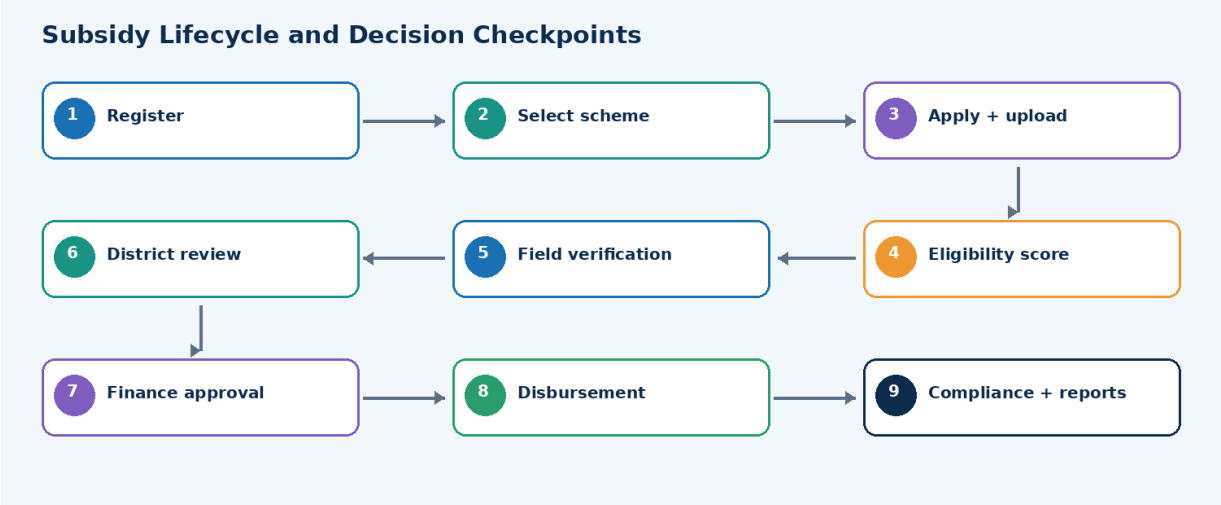
7.1 Validation sequence

- Load the selected scheme's eligibility configuration.
- Check hard gates: age/income and scheme-specific conditions such as student, farmer, female applicant, disability or rural residence.
- Validate required beneficiary profile fields.
- Check mandatory documents and clearly identify missing evidence.
- Calculate the profile and document score using configured weights.
- Return score, eligibility result and actionable missing-item feedback.
- Repeat server-side validation at final submission.

7.2 Explainability and fairness

- Transparent Results: The system explains which eligibility criteria are passed, failed, or still incomplete.
- Clear Feedback: Beneficiaries receive understandable information about missing documents or requirements so they can take corrective action.
- Consistent Rule Application: The same approved eligibility rules are applied consistently to applicants under thesame scheme.
- Boundary Testing: Eligibility rules are tested using different scenarios, such as income values below, equal to, and above the specified limit.
- Human Review and Accountability: Administrators can review eligibility assessments, while final decisions follow official scheme guidelines and authorized verification procedures.

8. Verification Workflow



Stage	Responsibility	Expected activity
Beneficiary submission	Beneficiary	Complete profile, select scheme, upload evidence and submit.
Eligibility evaluation	Application service / reviewer	Evaluate gates, score and document completeness.
Ground verification	Field officer	Check submitted information and record findings.
District review	District officer	Accept, reject or send back with remarks.
Finance approval	Finance approver	Confirm sanction decision and amount under configured rules.
Disbursement	Disbursement officer	Record release, UTR/reference, timestamp and remarks.
Compliance	Applicant / officer / reminders	Collect utilization evidence and flag overdue items.
Reporting	Authorized user	Review scheme/region activity and fund release.

8.1 Send-back, rejection and cooling period

When an application is returned for correction or rejected, the workflow should retain the decision and remarks. Project materials describe a 30-day cooling-period concept for applicable cases: a cooling-period end date prevents immediate resubmission; after expiry, the applicant can update evidence, eligibility can be recalculated and the resubmission count can increase. Applicant re-verification may remain active where required.

Confirm exact transition conditions against the current code and test date boundaries, repeated resubmissions and attempts to bypass the wait. The application should communicate the next eligible action and relevant date.

9. Disbursement Process

Disbursement is a controlled stage after the relevant review and finance approval. The platform can track staged releases and associate each release with supporting compliance requirements rather than treating approval as the end of the process.

9.1 Disbursement lifecycle

- Create a plan for an approved application with configured milestone amounts and due dates.

- Route the plan to the authorized disbursement role.
- Record release amount, transaction reference/UTR, timestamp and remarks.
- Update plan/application status so dashboards reflect released and remaining amounts.
- Track milestone-specific utilization evidence and compliance due dates.
- Generate reminders and flag overdue or incomplete evidence.

9.2 Controls and reconciliation

- Only authorized roles should approve or record releases.
- Keep sanctioned, planned and actually released amounts distinguishable.
- Capture transaction references for traceability and reconciliation.
- Audit changes to sanction and disbursement records.

10. Regional Hierarchy

The project includes region-aware workflow and budget concepts intended to route cases to responsible officers and summarize activity by area. Exact hierarchy labels and jurisdiction mapping should follow the data and access rules configured in the implementation.

Level / role	Typical responsibility	Access principle
Central / administrator	Configuration, officer management, monitoring and audit.	Authorized administrative scope.
State / regional administration	Review regional performance where configured.	Assigned region and permissions.
District officer	Review field findings and make district decisions.	Applications in assigned jurisdiction.
Field officer	Conduct ground verification.	Assigned cases / local area.
Finance approver	Approve sanction amounts within authority.	Financial actions restricted by role.
Disbursement officer	Record releases and payment references.	Release actions restricted by role.

10.1 Why regional controls matter

Jurisdiction-aware access reduces the chance of officers seeing or changing cases outside their responsibility. Regional reporting helps identify pending applications, workload, utilization and compliance issues. Test with accounts representing different roles and regions.

11. API Specifications

The application contains REST/API support for scheme, officer and application-related operations. Exact routes and request/response schemas should be taken from the controller annotations in the submitted code; the following is a controller mapping inventory found in the project package, where available.

Controller file	Mapping annotation
EligibilityController.java	@RequestMapping("/eligibility")
EligibilityController.java	@GetMapping("/officer/field")
EligibilityController.java	@GetMapping("/officer/district")
EligibilityController.java	@GetMapping("/officer/finance")
EligibilityController.java	@PostMapping("/officer/action")
AdminController.java	@RequestMapping("/admin")
AdminController.java	@GetMapping({"", "/dashboard"})
AdminController.java	@PostMapping("/schemes/save")
AdminController.java	@PostMapping("/schemes/delete/{id}")

Milestone3Controller.java	@RequestMapping("/milestone3")
Milestone3Controller.java	@GetMapping("/disbursement/{applicationId}")
Milestone3Controller.java	@PostMapping("/disbursement/{applicationId}/save")
Milestone3Controller.java	@PostMapping("/citizen/milestone/{milestoneId}/upload")
Milestone3Controller.java	@PostMapping("/disbursement/milestone/{milestoneId}/request")
Milestone3Controller.java	@GetMapping("/milestone/{milestoneId}/document")
Milestone3Controller.java	@PostMapping("/milestone/{milestoneId}/compliance")
Milestone3Controller.java	@PostMapping("/milestone/{milestoneId}/verify")
Milestone3Controller.java	@PostMapping("/milestone/{milestoneId}/release")
Milestone3Controller.java	@GetMapping("/citizen/milestone/{milestoneId}/receipt")
Milestone3Controller.java	@PostMapping("/disbursement/{applicationId}/hold")

11.1 API design expectations

- Use clear HTTP methods and stable, versioned paths where implemented.
- Validate request bodies and return useful client errors for invalid input.
- Require authentication for private beneficiary and officer data.
- Enforce role and jurisdiction checks for state-changing operations.
- Document required fields, status values, authorization rules and example resposes .

11.2 Security considerations

Do not expose government identifiers, uploaded document paths or audit details to unauthorized callers. Validate uploaded file type/size and avoid logging credentials or unnecessary personal data.

12. Integration Plan

The current project provides a foundation for application APIs and workflow tracking. External government, identity, payment or treasury connectivity should be treated as planned integration unless configured and successfully tested in the target environment.

Target	Purpose	Production validation
Identity/document verification	Confirm identity or validate evidence.	Approved provider, consent/legal basis, sandbox and failure handling.
Treasury/payment gateway	Send payment instructions and receive status.	API contract, idempotency, retries, reconciliation and security review.
SMS/email gateway	Notify about status, missing items and due dates.	Approved templates, delivery logs and privacy controls.
Scheme registry	Keep criteria/policy aligned.	Authoritative source, versioning and change history.
Analytics/export	Produce operational reports.	Validate calculations, permissions and export minimization.

13. Completion Status

Milestone	Scope	Completion summary
1	Environment, data foundations, application structure	Core project foundation and persistence structure established.
2	Beneficiary/scheme master, eligibility and verification	Scheme-aware validation, score feedback and multi-role workflow represented.
3	Disbursement, compliance and reminders	Staged release, due dates, compliance tracking and reminders represented.
4	Security, audit, APIs, analytics and documentation	Role-aware access, audit/API/reporting foundations represented; external connectivity depends on environment setup.

13.1 Final demonstration checklist

- Test beneficiary and officer roles.
- Open a scheme and inspect criteria and required documents.
- Submit a complete application and demonstrate score feedback.
- Test missing mandatory evidence and a failed eligibility gate.
- Demonstrate field verification, district review and send-back/rejection.
- Verify cooling-period and resubmission behavior with test dates.
- Demonstrate finance approval, disbursement record and UTR capture.
- Review compliance/reminders, audit history and analytics/reporting.
- Confirm unauthorized roles cannot access restricted actions or records.

14. Challenges and Solutions

Challenge	Response	Validation focus
Different scheme rules	Scheme configuration and centralized scoring/validation.	Age/income boundaries and scheme gates.
Incomplete evidence	Document requirements and actionable feedback.	Missing/invalid file and status updates.
Multi-role process	Sequential stages and role-specific queues.	Allowed transitions and unauthorized actions.
Repeated resubmission	Cooling-period date and resubmission metadata where configured.	Date boundaries, expiry and repeated attempts.
Fund traceability	Plan milestones, UTR/reference and timestamps.	Sanction vs release totals and reconciliation.
Sensitive information	Authentication, role checks and password hashing.	Endpoint access, region isolation and secrets.
External dependencies	Explicit API contracts and environment configuration.	Timeouts, retries, monitoring and audit.

15. Conclusion

SAGAR demonstrates a connected approach to subsidy and grant administration: beneficiaries provide information and evidence, the system evaluates scheme rules, authorized officers review applications, finance decisions move approved cases toward disbursement, and compliance/reporting support oversight.

The four milestone workstreams establish a foundation for a more traceable lifecycle: technical setup and data structures; eligibility and verification; disbursement and compliance; and security, audit, APIs and analytics. The illustrative beneficiary story explains the human motivation—reducing uncertainty and making the next step easier to understand.

Key outcomes

- Centralized scheme and beneficiary records.
- Structured eligibility checks and score feedback.
- Role-aware verification and review stages.
- Cooling-period and resubmission handling for applicable cases.
- Staged disbursement tracking and compliance follow-up.
- Audit and reporting foundations for accountability.